
Documentație Proiect PLF

Sistem Expert: Recomandare Carieră în Prolog

Echipa de Dezvoltare:

Tudor-Ioan Pirău

Mădălin Cazan

Ștefan Bălănică

Cătălin Buta

Repository GitHub:

<https://github.com/c-madalin/CareerRecommendation.git>

Martie 2026

Rezumat

Acest document prezintă arhitectura, designul și implementarea unui sistem expert bazat pe reguli, dezvoltat în limbajul de programare logică Prolog. Scopul principal al aplicației este de a oferi asistență decizională în orientarea profesională. Sistemul evaluează un set de input-uri ale utilizatorului (abilități tehnice/soft și interese personale), aplică un algoritm de potrivire procentuală și utilizează un model matematic ponderat pentru a ierarhiza cele mai potrivite cariere dintr-o bază de cunoștințe predefinită. Proiectul demonstrează aplicabilitatea paradigmei programării declarative în rezolvarea problemelor de tip "pattern matching" și raționament deductiv.

Cuprins

1	Introducere și Context Teoretic	3
1.1	Obiectivele Proiectului	3
2	Arhitectura Sistemului Expert	3
2.1	Baza de Cunoștințe (Knowledge Base - KB)	3
3	Logica de Inferență și Calcul Matematic	4
3.1	Algoritmul de Căutare a Elementelor Comune	4
3.2	Modelul Matematic de Evaluare	4
4	Descrierea Detaliată a Predicatelor Principale	5
4.1	Predicatul <code>recomanda_scor/3</code>	5
4.2	Predicate de Interfață și Filtrare Avansată	5
4.3	Extinderea Motorului: Filtrarea după Educație și Salariu	6
5	Scenarii de Testare și Validare (Trace-uri)	6
5.1	Cazul de Test 1: Profil Tehnic Ideal	6
5.2	Cazul de Test 2: Căutare cu Filtru Restrictiv (Domeniul Juridic)	7
5.3	Cazul de Test 3: Filtrare Avansată (Studii și Venit)	7
6	Metodologie de Dezvoltare și Contribuția Echipei	8
7	Concluzii și Direcții Viitoare de Dezvoltare	8

1 Introducere și Context Teoretic

Sistemele Expert (SE) reprezintă o ramură fundamentală a Inteligenței Artificiale, fiind proiectate pentru a emula capacitatea de luare a deciziilor a unui expert uman într-un domeniu restrâns și specific. Spre deosebire de programele procedurale clasice, care execută o secvență liniară de instrucțiuni, sistemele expert separă în mod clar cunoștințele despre domeniu (Baza de Cunoștințe) de mecanismul de procesare a acestora (Motorul de Inferență).

Limbaajul Prolog (Programming in Logic) este instrumentul ideal pentru o astfel de arhitectură, datorită mecanismelor sale native de unificare, backtracking (căutare cu revenire) și rezoluție.

1.1 Obiectivele Proiectului

Prezentul proiect își propune dezvoltarea unui sistem expert pentru recomandarea carierelor, vizând următoarele obiective specifice:

- Construirea unei Baze de Cunoștințe (Knowledge Base) robuste, cuprinzând minim 10 profiluri profesionale distincte, modelate prin attribute multiple (domeniu, abilități, interese, nivel de educație, salariu).
- Implementarea unui Motor de Inferență (Inference Engine) capabil să analizeze datele de intrare și să deducă compatibilitățile.
- Trecerea de la o logică booleană simplă (potrivire binară: Da/Nu) la un sistem calitativ, bazat pe calcul procentual.
- Aplicarea unei formule matematice ponderate pentru a genera un sistem de scor (Scoring System).
- Ierarhizarea rezultatelor și asigurarea unei interfețe prietenoase care tratează corect excepțiile (ex. lipsa totală a potrivirilor).

2 Arhitectura Sistemului Expert

Aplicația noastră respectă structura clasică a unui sistem expert bazat pe reguli de producție, fiind împărțită în următoarele module logice:

2.1 Baza de Cunoștințe (Knowledge Base - KB)

Baza de cunoștințe este complet declarativă și stochează faptele (facts) despre universul discursului. Am optat pentru o granularitate ridicată, împărțind entitatea "carieră" în mai multe predicate relaționale pentru a respecta forma normală și a evita redundanța datelor.

Faptele sunt definite astfel:

```
1 % cariera(Nume, Domeniu, Industrie)
2 cariera(programator_software, it, tehnologie).
3 cariera(medic, medicina, sanatate).
4
5 % abilitati(Cariera, ListaAbilitati)
6 abilitati(programator_software, [programare, logica, python, java, sql
7     ]).
8 % interese(Cariera, ListaInterese)
```

```

9 interese(programator_software, [tehnologie, inovatie, logica,
   calculatoare]).
10
11 % educatie(Cariera, NivelStudii)
12 educatie(programator_software, licenta).
13
14 % salariu(Cariera, IntervalSalarial)
15 salariu(programator_software, [5000, 15000]).

```

Listing 1: Fragment din Baza de Cunoștințe

Utilizarea listelor (ex. [programare, logica, python]) pentru atributele cu valori multiple este o alegere de design crucială. Aceasta permite aplicarea ulterioară a algoritmilor de intersecție a multimilor și calcularea gradului de suprapunere între profilul utilizatorului și cerințele profesiei.

3 Logica de Inferență și Calcul Matematic

Motorul de inferență este nucleul decizional al aplicației. Spre deosebire de un filtru de bază de date, motorul nostru realizează un calcul de similitudine.

3.1 Algoritmul de Căutare a Elementelor Comune

Pentru a determina gradul de acoperire, sistemul trebuie să numere elementele aflate în intersecția a două liste (lista utilizatorului și lista carierei). Pentru aceasta, am definit predicatul `numara_comune/3`.

```

1 numara_comune([], _, 0).
2 numara_comune([H|T], Lista2, Scor) :-
3     member(H, Lista2),
4     !,
5     numara_comune(T, Lista2, S1),
6     Scor is S1 + 1.
7 numara_comune([_|T], Lista2, Scor) :-
8     numara_comune(T, Lista2, Scor).

```

Listing 2: Predicatul de numărare cu operatorul Cut

Explicație (!): Operatorul cut este utilizat în a doua clauză pentru a eficientiza backtracking-ul. Dacă elementul H (Head) a fost găsit în Lista2 prin predicatul `member/2`, nu mai are rost ca Prolog să caute alte soluții pentru același element sau să evalueze a treia clauză (care tratează cazul în care elementul nu se află în listă). Aceasta transformă recursivitatea într-una deterministă, economisind memorie pe stiva de execuție.

3.2 Modelul Matematic de Evaluare

Sistemul nu lucrează cu numere absolute, ci cu procente, deoarece carierele pot avea un număr diferit de cerințe (unele necesită 5 abilități, altele 8). Pentru o comparare corectă, am implementat `calcul_procent/3`:

```

1 calcul_procent(ListaU, ListaCariera, Procent) :-
2     numara_comune(ListaU, ListaCariera, NumarComune),
3     length(ListaCariera, Total),
4     Total > 0,
5     Procent is NumarComune / Total.

```

Mai departe, se aplică modelul de decizie multicriterială pe baza următoarei formule:

$$Scor_{final} = (P_a \times Match_{abilitati}) + (P_i \times Match_{interese})$$

Unde:

- P_a reprezintă ponderea abilităților, setată la 0.6 (60%). S-a considerat că abilitățile concrete sunt ușor mai relevante în angajabilitate decât interesele generale.
- P_i reprezintă ponderea intereselor, setată la 0.4 (40%).

4 Descrierea Detaliată a Predicatelor Principale

4.1 Predicatul recomanda_scor/3

Acesta este predicatul principal de unificare care leagă faptele de logica matematică.

```
1 recomanda_scor(AU, IU, RecomandariSortate) :-  
2     findall(Scor - C,  
3         (cariera(C, _, _),  
4         abilitati(C, AC),  
5         interese(C, IC),  
6         calcul_procent(AU, AC, MatchA),  
7         calcul_procent(IU, IC, MatchI),  
8         Scor is (0.6 * MatchA) + (0.4 * MatchI),  
9         Scor > 0),  
10        RezultateNesortate),  
11    keysort(RezultateNesortate, SortateCrescator),  
12    reverse(SortateCrescator, RecomandariSortate).
```

Listing 3: Predicatul central de unificare și sortare

Mecanismul de funcționare:

1. Se utilizează meta-predicatul `findall/3` pentru a colecta toate instanțierile valide ale structurii `Scor - Carieră`. Acesta previne oprirea sistemului la prima soluție găsită și forțează evaluarea întregului spațiu al stărilor.
2. Condiția `Scor > 0` acționează ca un filtru de relevanță, excluzând din listă carierele cu care utilizatorul nu are absolut nicio afinitate.
3. Se folosește `keysort/2`. În Prolog, o structură de tip `Cheie-Valoare` (în cazul nostru `Scor-Cariera`) este sortată nativ după cheie, în ordine crescătoare.
4. Aplicăm `reverse/2` pentru a obține ordinea descrescătoare (cel mai mare scor primul), creând astfel topul recomandărilor.

4.2 Predicate de Interfață și Filtrare Avansată

Pentru a îmbunătăți experiența utilizatorului (User Experience), sistemul include rutine de tratare a cazurilor limită (Edge Cases).

```

1 afiseaza_recomandari([]) :-
2     write('Nu am gasit nicio cariera potrivita pentru profilul tau.
3     Mai incearca!'), nl.
4 afiseaza_recomandari([Scor-Nume|T]) :-
5     format('Scor: ~2f | Cariera: ~w~n', [Scor, Nume]),
6     afiseaza_restul(T).
7 afiseaza_restul([]).
8 afiseaza_restul([Scor-Nume|T]) :-
9     format('Scor: ~2f | Cariera: ~w~n', [Scor, Nume]),
10    afiseaza_restul(T).

```

Listing 4: Tratarea excepțiilor și parcurgerea listei

Prin introducerea predicatului `afiseaza_restul/1`, sistemul separă logic validarea inițială a listei de iterarea propriu-zisă. Astfel, mesajul de eroare este afișat exclusiv atunci când motorul de inferență nu a produs nicio potrivire (lista inițială este vidă), eliminând eroarea afișării acestuia la finalul unei liste valide.

4.3 Extinderea Motorului: Filtrarea după Educație și Salariu

Pentru a valorifica întreaga complexitate a Bazei de Cunoștințe, am dezvoltat un modul de filtrare avansată care extrage plafonul salarial superior și validează nivelul studiilor.

```

1 recomanda_filtru_avansat(AU, IU, EducatieDorita, SalariuMinim) :-
2     recomanda_scor(AU, IU, ToateRecomandarile),
3     findall(S - C,
4         (member(S - C, ToateRecomandarile),
5          educatie(C, EducatieDorita),
6          salariu(C, [_MinSalariu, MaxSalariu]),
7          MaxSalariu >= SalariuMinim),
8         RecomandariFiltrate),
9     write('--- RECOMANDARI FILTRATE ---'), nl,
10    write('Educatie necesara: '), write(EducatieDorita),
11    write(' | Salariu potential minim: '), write(SalariuMinim), nl,
12    afiseaza_recomandari(RecomandariFiltrate).

```

Listing 5: Filtrarea avansată după attribute complexe

Acest predicat demonstrează capacitatea limbajului Prolog de a face *pattern matching* avansat pe structuri de date. Extragerea valorii `MaxSalariu` din lista `[_MinSalariu, MaxSalariu]` utilizează o variabilă anonimă pentru limita inferioară, optimizând astfel memoria alocată căutării.

5 Scenarii de Testare și Validare (Trace-uri)

Validarea sistemului expert a fost realizată printr-o serie de teste de integrare.

5.1 Cazul de Test 1: Profil Tehnic Ideal

Scop: Verificarea capacității de ierarhizare corectă a carierelor similare. **Input:** Utilizatorul deține abilități de programare, python, și logică, având interese în tehnologie și inovație.

```

1 ?- start_recomandare([programare, python, logica], [tehnologie,
    inovatie]).

```

Rezultat pe consolă:

```
=== TOP RECOMANDARI PENTRU TINE ===  
Scor: 0.52 | Cariera: programator_software  
Scor: 0.28 | Cariera: analist_date  
Scor: 0.16 | Cariera: marketer_digital
```

Analiza rezultatului: Algoritmul funcționează conform așteptărilor. Deși analistul de date folosește *python*, scorul său este mai mic deoarece îi lipsesc competențele de statistică și R din lista utilizatorului.

5.2 Cazul de Test 2: Căutare cu Filtru Restrictiv (Domeniul Juridic)

Scop: Testarea predicatului `recomanda_filtru_domeniu/3`. **Input:** Abilități: oratorie, argumentare. Interese: lege, societate. Restricție domeniu: drept.

```
1 ?- recomanda_filtru_domeniu([oratorie, argumentare], [lege, societate  
    ], drept).
```

Rezultat pe consolă:

```
--- RECOMANDARI IN DOMENIUL: drept ---  
Scor: 0.46 | Cariera: avocat
```

5.3 Cazul de Test 3: Filtrare Avansată (Studii și Venit)

Scop: Testarea predicatului `recomanda_filtru_avansat/4` și integrarea atributelor complexe. **Input:** Abilități: statistica, python. Interese: cifre, tehnologie. Educație: master. Salariu potențial dorit: 10000.

```
1 ?- recomanda_filtru_avansat([statistica, python], [cifre, tehnologie],  
    master, 10000).
```

Rezultat pe consolă:

```
--- RECOMANDARI FILTRATE ---  
Educatie necesara: master | Salariu potential minim: 10000  
Scor: 0.40 | Cariera: analist_date
```

Analiza rezultatului: Deși setul de interese s-ar putea potrivi parțial și altor cariere din IT, sistemul a reținut exclusiv *analist_date*, deoarece este singura care îndeplinește simultan condiția educațională (master) și pragul salarial maxim (≥ 10000).

6 Metodologie de Dezvoltare și Contribuția Echipei

Dezvoltarea proiectului s-a realizat folosind o metodologie incrementală, bazată pe controlul versiunilor, asigurând o integrare organică a componentelor. Responsabilitățile au fost clar delimitate:

1. **Tudor-Ioan Pirău (Motorul de Inferență și Ierarhizarea):** A transformat regulile brute într-un sistem expert. A implementat formula de ponderare (60/40) și a gestionat agregarea stărilor folosind funcțiile native de sortare a dicționarilor (`keysort`) în Prolog. Conceput și scris Documentației. Code Review.
2. **Mădălin Cazan (Arhitectura Datelor):** A proiectat Baza de Cunoștințe. A asigurat consistența sintactică a faptelor, adăugând attribute complexe (inclusiv nivel educațional și scale salariale) pentru a acoperi exhaustiv cerințele din barem.
3. **Ștefan Bălănică (Logica Algoritmică):** A conceput și implementat algoritmi de parcurgere a listelor. Crearea predicatelor `numara_comune/3` (cu managementul tăieturii !) și `calcul_procent/3` a pus bazele matematice ale sistemului.
4. **Cătălin Buta (UI și Validarea Datelor):** A preluat datele brute de la motorul de inferență și a construit interfața de output. A scris handler-ele pentru excepții (reparând bug-ul listelor vide prin funcții recursive distincte) și a implementat mecanismele de rafinare a rezultatelor (filtre pe domeniu, educație și prag salarial).

7 Concluzii și Direcții Viitoare de Dezvoltare

Proiectul implementat reprezintă o demonstrație completă a puterii limbajului Prolog în dezvoltarea sistemelor de asistență decizională. Abordarea cantitativă (sistemul de scor procentual) oferă un avantaj major față de sistemele de filtrare strict logice, permițând ierarhizarea preferințelor în situații de incertitudine sau informații incomplete (utilizatorul nu deține toate abilitățile cerute, dar se apropie de profil).

Posibile dezvoltări viitoare (Future Work):

- Integrarea unei logici fuzzy (Fuzzy Logic) pentru a permite utilizatorilor să declare nivelul de competență (ex: python - avansat, java - începător), influențând astfel și mai precis calculul scorului.
- Extinderea bazei de date folosind web scraping pentru a actualiza automat interesele și cerințele profesionale de pe site-urile de recrutare.
- Dezvoltarea unei interfețe grafice (GUI) sau conectarea sistemului Prolog la un frontend web prin intermediul unui API (ex. SWI-Prolog HTTP server).